

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



CO3084 - Mật mã học và Mã hóa Thông tin (TN)

Problem Set 8

Zero-Knowledge Proofs

SVTH: Nguyễn Tấn Tài - 2212990

TP. Hồ Chí Minh, Tháng 10/2025

Mục lục

1	Hiểu về các thuộc tính Zero-Knowledge	1
1.1	(1) [20 điểm] Giao thức Schnorr	1
1.1.1	(a) [10 điểm] Phân tích giao thức	1
1.1.2	(b) [10 điểm] Mô phỏng và thuộc tính Zero-Knowledge	2
1.2	(2) [15 điểm] Tính Soundness thông qua trích xuất (Extraction)	3
2	Giao thức Sigma và sự kết hợp (Sigma Protocols and Composition)	5
2.1	(1) [20 điểm] Xây dựng các chứng minh phức tạp	5
2.1.1	(a) [10 điểm] Chứng minh OR (OR Proofs)	5
2.1.2	(b) [10 điểm] Chứng minh AND và tính bằng nhau (AND Proofs and Equality)	6
3	Chứng minh phi tương tác và phép biến đổi Fiat–Shamir	8
3.1	(1) [20 điểm] Phép biến đổi Fiat–Shamir (Fiat–Shamir Transform)	8
3.1.1	(a) [10 điểm] Từ tương tác sang phi tương tác (From Interactive to Non-Interactive)	8
3.1.2	(b) [10 điểm] Bảo mật trong mô hình Random Oracle	9
3.2	(2) [15 điểm] Mạch và Zero-Knowledge tổng quát (Circuits and General-Purpose ZK)	10
4	Ứng dụng thực tế (Real-World Applications)	11
4.1	(1) [10 điểm] Các hệ thống bảo vệ quyền riêng tư (Privacy-Preserving Systems)	11
4.1.1	(a) [5 điểm] Giải thích một giao dịch “shielded” trong Zcash chứng minh điều gì mà không tiết lộ điều gì	11
4.1.2	(b) [5 điểm] Zcash đã tiến hóa từ Sprout → Sapling → Halo 2 (không cần thiết lập tin cậy)	11
4.2	Bonus Question (10 điểm cộng)	12

1 Hiểu về các thuộc tính Zero-Knowledge

1.1 (1) [20 điểm] Giao thức Schnorr

1.1.1 (a) [10 điểm] Phân tích giao thức

Bạn đang triển khai xác thực cho một ứng dụng nhắn tin an toàn sử dụng giao thức nhận dạng Schnorr. Alice có khóa công khai $A = g^a$, trong đó a là khóa bí mật của cô.

Trả lời:

i. Mô phỏng một lượt thực thi hoàn chỉnh của giao thức Schnorr

Hiển thị toàn bộ các phép tính khi:

- Alice chọn $y = 42$
- Bên xác minh gửi thách thức $c = 7$
- Khóa bí mật của Alice là $a = 13$

(sử dụng modulo 101 để đơn giản). Xác minh rằng giao thức được chấp nhận.

Bài làm:

Dưới đây là quá trình mô phỏng chi tiết từng bước của giao thức Schnorr giữa Alice (Prover) và Verifier:

Bước 1: Commitment (Cam kết) Alice chọn số ngẫu nhiên $y = 42$. Cô ấy tính giá trị cam kết Y để gửi cho Verifier. Giả sử g là phần tử sinh của nhóm (đề bài không cho giá trị cụ thể của g , nhưng ta vẫn thực hiện các bước tính toán trên số mũ).

$$Y = g^y \pmod{101} = g^{42} \pmod{101}$$

Alice gửi Y cho Verifier.

Bước 2: Challenge (Thách thức) Verifier nhận được cam kết Y , sau đó chọn một số ngẫu nhiên $c = 7$ và gửi lại cho Alice làm thách thức.

Bước 3: Response (Phản hồi) Alice nhận thách thức $c = 7$. Cô ấy sử dụng khóa bí mật $a = 13$ và số ngẫu nhiên $y = 42$ đã chọn để tính phản hồi r :

$$r = y + c \cdot a \pmod{n}$$

Thay số vào công thức (với n là bậc của nhóm, ở đây ta tính toán trên trường số nguyên modulo 101 cho đơn giản):

$$r = 42 + 7 \cdot 13 = 42 + 91 = 133$$

Thực hiện phép modulo 101:

$$r = 133 \pmod{101} = 32$$

Alice gửi giá trị $r = 32$ cho Verifier.

Bước 4: Verification (Kiểm tra) Verifier nhận được $r = 32$. Verifier kiểm tra tính hợp lệ của bằng chứng bằng cách so sánh hai giá trị g^r và $Y \cdot A^c$.

• **Về trái:** $g^r = g^{32}$.

• **Về phải:**

$$Y \cdot A^c = g^{42} \cdot (g^{13})^7 = g^{42} \cdot g^{91} = g^{42+91} = g^{133}$$

Trong modulo 101 (đối với số mũ là bậc của nhóm, thường là $n - 1$ hoặc một số nguyên tố q , nhưng ở đây ta xét tính tương đương):

$$g^{133} = g^{32} \pmod{101} \quad (\text{vì } 133 \equiv 32 \pmod{101})$$

Ta thấy $g^r \equiv Y \cdot A^c \pmod{g^{32}}$. **Kết luận:** Giao thức được chấp nhận (Accept).

ii. Giao thức yêu cầu Alice chọn một giá trị ngẫu nhiên mới y cho mỗi lần thực thi

Nếu Alice tái sử dụng cùng một y cho hai thách thức khác nhau c_1 và c_2 , loại tấn công nào sẽ trở nên khả thi? Hãy chỉ ra cách rút trích khóa bí mật của cô trong trường hợp đó.

Bài làm:

Nếu Alice tái sử dụng y (dẫn đến cùng cam kết Y) cho hai phiên xác thực khác nhau, cô ấy sẽ phải đối mặt với ****tấn công trích xuất khóa bí mật (Key Extraction Attack)****. Kẻ tấn công (đóng vai trò Verifier độc hại) có thể tính toán ngược ra khóa bí mật a của Alice một cách dễ dàng.

Cụ thể, giả sử Alice thực hiện hai lần chứng minh với cùng y :

- Lần 1: Nhận thách thức c_1 , trả về $r_1 = y + c_1 \cdot a \pmod{n}$.
- Lần 2: Nhận thách thức c_2 (với $c_1 \neq c_2$), trả về $r_2 = y + c_2 \cdot a \pmod{n}$.

Kẻ tấn công có hệ phương trình hai ẩn (y và a). Hẳn chỉ cần lấy hiệu của hai phản hồi:

$$r_1 - r_2 = (y + c_1 a) - (y + c_2 a) = a(c_1 - c_2) \pmod{n}$$

Vì $c_1 \neq c_2$, nên $(c_1 - c_2)$ khác 0 và có nghịch đảo modulo n . Kẻ tấn công tính khóa bí mật a bằng công thức:

$$a = (r_1 - r_2) \cdot (c_1 - c_2)^{-1} \pmod{n}$$

Như vậy, toàn bộ bảo mật của hệ thống bị phá vỡ chỉ vì việc tái sử dụng số ngẫu nhiên y .

1.1.2 (b) [10 điểm] Mô phỏng và thuộc tính Zero-Knowledge

Bob tuyên bố rằng anh đã xác thực được Alice bằng giao thức Schnorr và đưa cho Charlie bản ghi (transcript) (A, Y, c, r) làm bằng chứng.

Trả lời:

i. Dựa vào phép so sánh “Robin Hood và mũi tên” trong slide bài giảng, hãy giải thích vì sao Charlie không nên tin điều đó

Tại sao tình huống này giống với việc vẽ bia quanh mũi tên?

Bài làm:

Charlie tuyệt đối không nên tin vào bản ghi này vì tính chất ****Zero-Knowledge**** của giao thức Schnorr cho phép bất kỳ ai (kể cả Bob) cũng có thể tự tạo ra một bản ghi hợp lệ mà không cần Alice tham gia, và không cần biết khóa bí mật a .

Phép so sánh “Robin Hood và mũi tên” giải thích điều này rất hình tượng:

- ****Giao thức thật (Alice làm):**** Giống như Robin Hood bắn mũi tên trúng hồng tâm. Alice phải cam kết (bắn mũi tên Y) trước khi biết thách thức (vị trí bia c). Việc cô ấy bắn trúng (tính được r khớp) chứng tỏ cô ấy thực sự có tài năng (biết khóa a).
- ****Bản ghi giả mạo (Bob làm):**** Giống như việc một người bắn mũi tên vào bức tường trống trước (r), sau đó mới đến vẽ cái bia (Y) xung quanh mũi tên đó sao cho nó nằm ngay hồng tâm.

Trong toán học, Bob có thể chọn trước kết quả r và thách thức c , sau đó tính ngược ra giá trị cam kết $Y = g^r \cdot A^{-c}$. Bản ghi (Y, c, r) này hoàn toàn hợp lệ về mặt toán học nhưng không chứng minh được rằng cuộc hội thoại thực sự đã diễn ra với Alice. Do đó, nó không có giá trị bằng chứng với người thứ ba (non-transferable).

ii. Viết pseudocode (giả mã) cho một bộ mô phỏng (simulator) có thể sinh ra các transcript hợp lệ trông giống thật của giao thức Schnorr mà không cần biết khóa bí mật a

Bài làm:

Dưới đây là giả mã cho Simulator. Ý tưởng chính là thực hiện quy trình ngược: Chọn kết quả trước, tính nguyên nhân sau.

```
def simulate_schnorr_transcript(G, A, n):  
    """  
    Tạo ra bản ghi Schnorr giả mạo (Y, c, r) hợp lệ  
    mà KHÔNG cần biết khóa bí mật a.  
    Input:  
        G: Phần tử sinh  
        A: Khóa công khai của Alice (A = g^a)  
        n: Bậc của nhóm  
    Output:  
        Transcript (Y, c, r) thỏa mãn g^r = Y * A^c  
    """  
  
    # Bước 1: Chọn ngẫu nhiên thách thức c và phản hồi r trước  
    # (Đây là hành động "bắn mũi tên vào tường trống")  
    c = random_int(0, n)  
    r = random_int(0, n)  
  
    # Bước 2: Tính ngược cam kết Y để thỏa mãn phương trình xác minh  
    # Ta cần: g^r = Y * A^c  
    # Suy ra: Y = g^r * (A^c)^(-1) = g^r * A^(-c)  
    # (Đây là hành động "vẽ bia xung quanh mũi tên")  
    term1 = pow(G, r, n)          # g^r  
    term2 = mod_inverse(pow(A, c, n), n) # A^(-c)  
    Y = (term1 * term2) % n  
  
    return (Y, c, r)
```

iii. Một đồng nghiệp đề xuất “tối ưu” giao thức Schnorr bằng cách để bên chứng minh gửi trực tiếp y thay vì $Y = g^y$

Giải thích tại sao điều này phá vỡ thuộc tính zero-knowledge.

Bài làm:

Đề xuất này cực kỳ nguy hiểm và phá hủy hoàn toàn tính bảo mật của giao thức. Nếu Alice gửi trực tiếp giá trị ngẫu nhiên y (thay vì $Y = g^y$), thì sau khi gửi phản hồi $r = y + c \cdot a$, Verifier (hoặc bất kỳ ai nghe lén được) có thể tính ngay ra khóa bí mật a bằng phép tính đơn giản:

$$c \cdot a = r - y \pmod{n}$$

$$a = (r - y) \cdot c^{-1} \pmod{n}$$

Lúc này, Alice không còn giữ được bí mật nào cả. Thuộc tính **Zero-Knowledge** (không tiết lộ tri thức) bị vi phạm nghiêm trọng vì Verifier học được toàn bộ khóa bí mật của Prover. Việc sử dụng $Y = g^y$ (bài toán logarit rời rạc) là lớp bảo vệ thiết yếu để che giấu y và qua đó bảo vệ a .

1.2 (2) [15 điểm] Tính Soundness thông qua trích xuất (Extraction)

Bạn chặn được hai transcript được chấp nhận từ cùng một bên chứng minh, với cùng cam kết (commitment) nhưng thách thức khác nhau:

$(Y = g^{100}, c = 15, r = 250)$ và $(Y = g^{100}, c' = 22, r' = 341)$
Khóa công khai là $A = g^{17}$.

Trả lời:

(a) [10 điểm] Trích xuất khóa bí mật a từ hai transcript trên

Trình bày rõ các bước tính toán.

Bài làm:

Vì hai transcript có cùng cam kết Y (tức là cùng một y ẩn), ta có thể áp dụng phương pháp giải hệ phương trình tuyến tính để tìm a .

Ta có hai phương trình phản hồi:

$$\begin{cases} 250 = y + 15 \cdot a \pmod{n} & (1) \\ 341 = y + 22 \cdot a \pmod{n} & (2) \end{cases}$$

Lấy phương trình (2) trừ cho phương trình (1) để triệt tiêu y :

$$341 - 250 = (y + 22a) - (y + 15a) \pmod{n}$$

$$91 = 7a \pmod{n}$$

Để tìm a , ta chia cả hai vế cho 7 (hoặc nhân với nghịch đảo của 7 modulo n). Giả sử ta đang làm việc trong trường số học thông thường hoặc n đủ lớn để không bị wrap-around ở các giá trị nhỏ này:

$$a = \frac{91}{7} = 13$$

Vậy khóa bí mật là $**a = 13**$. (Ta có thể kiểm tra lại: $A = g^{17}$ trong đề bài có thể là lỗi đánh máy hoặc một giá trị khác, nhưng dựa trên dữ liệu transcript r và c , kết quả tính toán ra $a = 13$. Nếu $A = g^{17}$ là đúng thì transcript này là giả mạo vì a phải là 17. Tuy nhiên, với yêu cầu "trích xuất từ transcript", kết quả $a = 13$ là đáp án chính xác dựa trên dữ liệu đầu vào).

(b) [5 điểm] Giải thích tại sao kỹ thuật trích xuất này cho thấy rằng bên chứng minh gian lận chỉ có xác suất tối đa $\frac{1}{n}$ để lừa được bên xác minh

Bài làm:

Kỹ thuật trích xuất (Extraction) chứng minh tính chất ****Soundness**** (Tính đúng đắn) của giao thức. Lập luận như sau:

- Nếu một Prover có thể trả lời đúng cho *ít nhất hai* thách thức khác nhau c và c' với cùng một cam kết Y , thì như ta vừa tính ở trên, Prover đó chắc chắn phải biết (hoặc tính được) khóa bí mật a .
- Do đó, nếu một Prover **không** biết khóa a (kẻ gian lận), hắn chỉ có thể chuẩn bị sẵn câu trả lời đúng cho **tối đa một** giá trị thách thức c duy nhất (bằng cách chọn trước c và r rồi tính Y).
- Khi Verifier chọn một thách thức ngẫu nhiên từ không gian n giá trị, xác suất để Verifier chọn trúng ngay cái thách thức duy nhất mà kẻ gian lận đã chuẩn bị là $\frac{1}{n}$.

Với n đủ lớn (ví dụ 2^{256}), xác suất $\frac{1}{n}$ là cực kỳ nhỏ, coi như bằng 0. Điều này đảm bảo rằng kẻ gian lận hầu như không thể đánh lừa được Verifier.

2 Giao thức Sigma và sự kết hợp (Sigma Protocols and Composition)

2.1 (1) [20 điểm] Xây dựng các chứng minh phức tạp

2.1.1 (a) [10 điểm] Chứng minh OR (OR Proofs)

Bạn đang xây dựng một hệ thống tố giác ẩn danh (anonymous whistleblower system), trong đó người dùng có thể chứng minh rằng họ hoặc là nhân viên hiện tại (có khóa công khai A_1) hoặc là cựu nhân viên hợp lệ (có khóa công khai A_2) — mà không tiết lộ họ thuộc trường hợp nào.

Trả lời:

i. Thiết kế một chứng minh dạng OR cho kịch bản này

Giả sử người tố giác biết khóa riêng a_1 tương ứng với A_1 . Hãy mô tả cách họ tạo ra bằng chứng khi bên xác minh đưa ra thách thức $C = 50$.

Bài làm:

Đây là bài toán chứng minh kiến thức về một trong hai bí mật (1-out-of-2 Proof). Vì người tố giác biết a_1 (nhân viên hiện tại) nhưng không biết a_2 , họ sẽ thực hiện quy trình sau:

1. **Bước 1: Mô phỏng (Simulate) cho nhánh không biết (A_2):** Vì không biết a_2 , người tố giác không thể thực hiện giao thức thật. Họ phải "gian lận" hợp lệ bằng cách chọn trước kết quả.

- Chọn ngẫu nhiên thách thức giả c_2 và phản hồi giả r_2 .
- Tính cam kết giả Y_2 sao cho khớp với phương trình xác minh:

$$Y_2 = g^{r_2} \cdot A_2^{-c_2} \pmod{n}$$

2. **Bước 2: Thực hiện thật (Real Execution) cho nhánh biết (A_1):**

- Chọn một số ngẫu nhiên y_1 (bí mật tạm thời).
- Tính cam kết thật: $Y_1 = g^{y_1} \pmod{n}$.

3. **Bước 3: Tính thách thức cho nhánh thật:** Tổng hai thách thức phải bằng thách thức chủ C do Verifier gửi ($C = 50$).

- Tính $c_1 = C - c_2 \pmod{n}$.
- Vì c_2 đã chọn ngẫu nhiên ở Bước 1, nên c_1 cũng sẽ mang tính ngẫu nhiên, đảm bảo an toàn.

4. **Bước 4: Tính phản hồi cho nhánh thật:** Sử dụng khóa bí mật a_1 để tính phản hồi Schnorr chuẩn:

- $r_1 = y_1 + c_1 \cdot a_1 \pmod{n}$.

5. **Bước 5: Gửi bằng chứng:** Người tố giác gửi bộ bằng chứng gồm: $(Y_1, Y_2, c_1, c_2, r_1, r_2)$ cho Verifier.

Verifier sẽ kiểm tra 3 điều kiện: 1. $c_1 + c_2 = C \pmod{n}$ 2. $g^{r_1} \equiv Y_1 \cdot A_1^{c_1} \pmod{n}$ 3. $g^{r_2} \equiv Y_2 \cdot A_2^{c_2} \pmod{n}$

ii. Giải thích vì sao bên xác minh không thể biết liệu người chứng minh biết a_1 hay a_2

Sử dụng khái niệm về bản ghi mô phỏng (simulated transcripts) so với bản ghi thật (real transcripts).

Bài làm:

Bên xác minh (Verifier) hoàn toàn không thể phân biệt được đâu là nhánh "thật" và đâu là nhánh "mô phỏng" vì tính chất **Perfect Zero-Knowledge** của giao thức Schnorr.

Cụ thể:

- Một bản ghi Schnorr được tạo ra bởi người biết khóa (Real Transcript: $Y = g^y, c, r = y + ca$) có phân phối xác suất thống kê giống hệt với một bản ghi được tạo ra bởi bộ mô phỏng (Simulated Transcript: chọn c, r trước, tính $Y = g^r A^{-c}$).

- Trong chứng minh OR ở trên, ta có hai bộ (Y_1, c_1, r_1) và (Y_2, c_2, r_2) . Một bộ là thật, một bộ là mô phỏng. Nhưng vì cả hai đều trông "ngẫu nhiên" và hợp lệ như nhau, Verifier không có manh mối nào để biết a_1 hay a_2 đã được sử dụng.
- Điều này giống như việc nhìn vào hai bức ảnh chụp đích bắn cung: một bức là bắn thật trúng đích, một bức là vẽ bia quanh mũi tên. Nếu người vẽ bia vẽ khéo léo (đúng toán học), hai bức ảnh sẽ y hệt nhau.

Do đó, danh tính của người tổ giác (là nhân viên hiện tại hay cựu nhân viên) được giữ bí mật hoàn toàn.

2.1.2 (b) [10 điểm] Chứng minh AND và tính bằng nhau (AND Proofs and Equality)

Một sàn giao dịch tiền điện tử yêu cầu người dùng chứng minh rằng họ sở hữu tài khoản trên ba blockchain khác nhau, trong đó mỗi tài khoản dùng cùng một khóa riêng nhưng với các bộ sinh (generator) khác nhau:

$$A_1 = g_1^a, \quad A_2 = g_2^a, \quad A_3 = g_3^a$$

Trả lời:

i. Xây dựng một giao thức Sigma chứng minh rằng người dùng biết một a sao cho cả ba log rời rạc đều bằng nhau

Giải thích cách dùng chung một thách thức C cho cả ba chứng minh giúp duy trì tính soundness.

Bài làm:

Để chứng minh rằng cùng một khóa bí mật a được sử dụng cho cả ba tài khoản (Equality of Discrete Logarithms), ta sử dụng kỹ thuật ****AND Composition**** với một thách thức chung.

****Giao thức:****

1. **Commitment (Cam kết):** Prover chọn một số ngẫu nhiên y duy nhất. Tính ba cam kết tương ứng với ba phần tử sinh:

$$Y_1 = g_1^y, \quad Y_2 = g_2^y, \quad Y_3 = g_3^y$$

Gửi (Y_1, Y_2, Y_3) cho Verifier.

2. **Challenge (Thách thức):** Verifier gửi một thách thức ngẫu nhiên C duy nhất cho Prover.

3. **Response (Phản hồi):** Prover tính một phản hồi duy nhất:

$$r = y + C \cdot a \pmod{n}$$

Gửi r cho Verifier.

4. **Verification (Kiểm tra):** Verifier kiểm tra đồng thời ba phương trình:

$$g_1^r \stackrel{?}{=} Y_1 \cdot A_1^C, \quad g_2^r \stackrel{?}{=} Y_2 \cdot A_2^C, \quad g_3^r \stackrel{?}{=} Y_3 \cdot A_3^C$$

****Tại sao dùng chung C lại quan trọng?*** Việc bắt buộc Prover phải trả lời một thách thức C giống nhau bằng một phản hồi r giống nhau cho cả ba phương trình chính là "sợi dây" buộc chặt ba khóa lại với nhau. Nếu Prover gian lận dùng a_1 cho cái đầu, a_2 cho cái thứ hai ($a_1 \neq a_2$), thì hắn sẽ cần tính $r = y + C a_1$ để thỏa mãn cái 1 và $r = y + C a_2$ để thỏa mãn cái 2. Điều này là vô lý vì r phải là duy nhất. Do đó, a_1 buộc phải bằng a_2 (và bằng a_3).

ii. Giải thích vì sao việc chạy ba chứng minh Schnorr độc lập (với các thách thức riêng biệt) sẽ không đạt được cùng mục tiêu

Bài làm:

Nếu chạy 3 giao thức Schnorr độc lập, Verifier sẽ gửi 3 thách thức khác nhau c_1, c_2, c_3 . Lúc này, Prover có thể gian lận dễ dàng:

- Prover có thể biết a_1 là log của A_1 , a_2 là log của A_2 , và a_3 là log của A_3 , nhưng a_1, a_2, a_3 là các số hoàn toàn khác nhau.
- Prover thực hiện:
 - Phiên 1: $r_1 = y_1 + c_1 a_1$ (thỏa mãn A_1)
 - Phiên 2: $r_2 = y_2 + c_2 a_2$ (thỏa mãn A_2)
 - Phiên 3: $r_3 = y_3 + c_3 a_3$ (thỏa mãn A_3)

Cả 3 phiên đều thành công, chứng minh Prover biết log của từng cái, NHƯNG không chứng minh được rằng $a_1 = a_2 = a_3$. Việc cho phép các thách thức (và phản hồi) riêng biệt đã cắt đứt mối liên kết ràng buộc giữa các bí mật. Chỉ có thách thức chung (Common Challenge) mới ép buộc được sự bằng nhau.

3 Chứng minh phi tương tác và phép biến đổi Fiat-Shamir

(Non-Interactive Proofs and Fiat-Shamir)

3.1 (1) [20 điểm] Phép biến đổi Fiat-Shamir (Fiat-Shamir Transform)

3.1.1 (a) [10 điểm] Từ tương tác sang phi tương tác (From Interactive to Non-Interactive)

Bạn đang chuyển đổi giao thức nhận dạng Schnorr thành một sơ đồ chữ ký số bằng phép biến đổi Fiat-Shamir.

Trả lời:

i. Trình bày toàn bộ thuật toán ký cho thông điệp

“Transfer \$1000 to Bob”

Bao gồm cách tính thách thức $c = H(A, Y, m)$.

Bài làm:

Để chuyển đổi giao thức Schnorr tương tác thành chữ ký số phi tương tác (Non-Interactive Digital Signature), ta áp dụng phép biến đổi Fiat-Shamir. Ý tưởng cốt lõi là thay thế vai trò của Verifier (người gửi thách thức ngẫu nhiên) bằng một hàm băm mật mã (Hash Function).

****Thuật toán ký (Signing Algorithm):**** Giả sử Alice có cặp khóa $(a, A = g^a)$ và muốn ký lên thông điệp $m = \text{“Transfer $1000 to Bob”}$.

- Bước 1: Tạo cam kết (Commitment)** Alice chọn một số ngẫu nhiên y (nonce). Tính cam kết $Y = g^y \pmod{n}$.
- Bước 2: Tự tính thách thức (Compute Challenge)** Thay vì đợi Verifier gửi c , Alice tự tính c bằng cách băm tất cả các dữ liệu công khai liên quan đến phiên làm việc này:

$$c = H(A, Y, m) = H(A, Y, \text{“Transfer $1000 to Bob”})$$

Việc này đảm bảo c phụ thuộc chặt chẽ vào thông điệp và cam kết, không thể thay đổi tùy tiện.

- Bước 3: Tính phản hồi (Compute Response)** Alice tính phản hồi r giống như trong giao thức tương tác:

$$r = y + c \cdot a \pmod{n}$$

- Bước 4: Xuất chữ ký (Output Signature)** Chữ ký cuối cùng cho thông điệp m là cặp giá trị $\sigma = (c, r)$ (hoặc đôi khi là (Y, r) tùy biến thể, nhưng (c, r) phổ biến hơn vì kích thước nhỏ gọn).

ii. Giải thích vì sao việc đưa thông điệp m vào trong hàm băm là cực kỳ quan trọng

Nếu thay vào đó sử dụng $c = H(A, Y)$, thì loại tấn công nào sẽ trở nên khả thi?

Bài làm:

Việc đưa thông điệp m vào đầu vào của hàm băm là ****bắt buộc**** để liên kết chữ ký với nội dung văn bản. Nếu ta loại bỏ m và chỉ tính $c = H(A, Y)$, hệ thống sẽ dính lỗ hổng nghiêm trọng gọi là ****Tấn công giả mạo sự tồn tại (Existential Forgery Attack)****.

****Kịch bản tấn công:****

- Giả sử Alice ký một thông điệp vô hại $m_1 = \text{“Hello”}$. Cô ấy tạo ra chữ ký (c, r) với $c = H(A, Y)$.
- Kẻ tấn công (Eve) bắt được chữ ký (c, r) này.
- Eve muốn giả mạo chữ ký của Alice cho thông điệp $m_2 = \text{“Transfer $1M to Eve”}$.
- Vì quy trình kiểm tra chữ ký chỉ là: $c \stackrel{?}{=} H(A, Y)$ và $g^r \stackrel{?}{=} Y \cdot A^c$.

- Eve nhận thấy rằng việc kiểm tra c hoàn toàn không phụ thuộc vào m . Do đó, chữ ký (c, r) hợp lệ cho m_1 cũng sẽ **hợp lệ cho bất kỳ thông điệp m_2 nào khác**.
- Eve chỉ cần copy-paste chữ ký của m_1 sang cho m_2 , và Verifier sẽ chấp nhận nó.

Do đó, m phải nằm trong hàm băm để mỗi chữ ký là duy nhất cho một nội dung cụ thể.

3.1.2 (b) [10 điểm] Bảo mật trong mô hình Random Oracle

Phép biến đổi Fiat–Shamir giả định rằng hàm băm hoạt động như một Random Oracle.

Trả lời:

i. Giải thích những thuộc tính của Random Oracle mà một hàm băm thực tế (như SHA-256) có thể không đạt được hoàn hảo

Bài làm:

****Random Oracle (RO)**** là một mô hình lý tưởng hóa trong toán học, giống như một "hộp đen" hoàn hảo:

- Với mỗi đầu vào mới, nó trả về một đầu ra ngẫu nhiên hoàn toàn (uniform random) từ không gian đích.
- Nếu gặp lại đầu vào cũ, nó trả về đúng kết quả đã tạo ra trước đó.
- Không ai có thể đoán trước hay tính toán đầu ra nếu không thực sự gọi hàm.

****Hàm băm thực tế (như SHA-256)**** khác biệt ở chỗ:

- ****Tính xác định (Deterministic) và có cấu trúc:**** SHA-256 là một thuật toán cụ thể, không phải ngẫu nhiên. Nếu bạn phân tích kỹ thuật toán, bạn có thể tìm thấy các mối quan hệ toán học giữa đầu vào và đầu ra (dù rất khó).
- ****Length Extension Attack:**** Với các hàm băm cấu trúc Merkle-Damgård (như SHA-256), nếu biết $H(x)$, ta có thể tính được $H(x||padding||y)$ mà không cần biết x . Random Oracle không cho phép điều này (đầu ra của chuỗi mới phải hoàn toàn ngẫu nhiên và không liên quan đến chuỗi cũ).
- ****Khả năng bị tấn công:**** Trong thực tế, đã có những trường hợp (như tấn công vào Fiat-Shamir khi dùng hàm băm yếu hoặc thiết kế sai) mà kẻ tấn công lợi dụng cấu trúc của hàm băm để tạo ra bằng chứng giả, điều không thể xảy ra với Random Oracle.

ii. Mô tả ở mức cao Bổ đề Forking (Forking Lemma)

Giải thích cách bổ đề này cho phép rút trích nhân chứng (witness) từ một kẻ tấn công có khả năng giả mạo chứng minh Fiat–Shamir.

Bài làm:

****Bổ đề Forking (Forking Lemma)**** là công cụ lý thuyết quan trọng để chứng minh độ an toàn của chữ ký Fiat-Shamir. Nó hoạt động dựa trên kỹ thuật "tua ngược thời gian" (rewinding) trong máy ảo.

****Cơ chế:****

- Giả sử có một kẻ tấn công (Adversary) có thể tạo ra chữ ký giả mạo (Y, c, r) với xác suất không nhỏ.
- Ta chạy kẻ tấn công lần đầu tiên. Khi nó hỏi Random Oracle để lấy giá trị băm cho (A, Y, m) , ta trả về thách thức c . Kẻ tấn công xuất ra chữ ký hợp lệ (c, r) .
- Sau đó, ta ****tua ngược**** (rewind) kẻ tấn công lại đúng thời điểm nó vừa gửi truy vấn băm đó. Mọi trạng thái bộ nhớ của nó được giữ nguyên như cũ.
- Lần này, ta trả về một thách thức ****khác**** c' (ngẫu nhiên mới). Do trạng thái nội tại giống hệt lần trước, kẻ tấn công (nếu đủ giỏi) vẫn sẽ tạo ra được chữ ký hợp lệ thứ hai (c', r') với cùng cam kết Y .

- Kết quả: Ta thu được hai bộ (Y, c, r) và (Y, c', r') . Đây chính là điều kiện để áp dụng kỹ thuật trích xuất (như đã làm ở bài 1) để tìm ra khóa bí mật a .

Tóm lại, Bỏ đề Forking cho thấy: Nếu ai đó giả mạo được chữ ký, họ thực sự phải biết khóa bí mật.

3.2 (2) [15 điểm] Mạch và Zero-Knowledge tổng quát (Circuits and General-Purpose ZK)

Bạn muốn chứng minh rằng mình biết một giá trị x sao cho $\text{SHA-256}(x) = y$ với một y đã cho.

Trả lời:

(a) [7.5 điểm] Giải thích vì sao việc tạo một giao thức Sigma riêng cho phát biểu này là rất khó khăn

Làm thế nào việc biểu diễn SHA-256 dưới dạng mạch (circuit) lại khiến chứng minh này trở nên khả thi?

Bài làm:

- ****Khó khăn của Sigma Protocol:**** Giao thức Sigma (như Schnorr) dựa trên các tính chất đại số đẹp đẽ của lý thuyết số (như $g^a \cdot g^b = g^{a+b}$). Trong khi đó, SHA-256 là một "mớ hỗn độn" các phép toán bitwise phi tuyến tính (XOR, AND, Rotate, Shift) được thiết kế để phá vỡ mọi cấu trúc đại số. Việc cố gắng xây dựng một phương trình đại số $Y = g^x$ tương đương cho hàng ngàn vòng lặp và phép toán bit của SHA-256 là cực kỳ phức tạp và gần như bất khả thi để làm thủ công.
- ****Giải pháp Mạch (Circuits):**** Thay vì tìm công thức toán học, ta "vẽ" lại toàn bộ quy trình tính toán của SHA-256 dưới dạng một mạch điện tử khổng lồ (Arithmetic Circuit hoặc Boolean Circuit). Mạch này gồm hàng ngàn cổng logic (cổng cộng, cổng nhân, cổng XOR).
- Các hệ thống ZK tổng quát (General-purpose ZK) như zk-SNARKs (sử dụng R1CS hoặc QAP) cho phép ta chứng minh rằng: "Tôi biết các tín hiệu đi vào dây dẫn sao cho tất cả các cổng trong mạch đều đúng". Máy tính có thể tự động chuyển đổi mạch SHA-256 thành hệ thống ràng buộc đa thức này mà không cần con người phải thiết kế giao thức riêng.

(b) [7.5 điểm] Ứng dụng của bạn cần chứng minh mệnh đề

"Tôi biết x sao cho $x^3 + x + 5 = 35$ trong $\mathbb{Z}_{101}$ "

Bạn sẽ chọn giao thức tùy chỉnh hay dạng mạch tổng quát? Hãy giải thích lựa chọn của bạn.

Bài làm:

Tôi sẽ chọn ****Giao thức tùy chỉnh (Custom Protocol)**** hoặc một biến thể Sigma Protocol đơn giản cho đa thức.

****Lý do:****

- ****Hiệu suất:**** Bài toán $x^3 + x - 30 = 0$ là một quan hệ đại số cực kỳ đơn giản và nhỏ gọn.
- ****Overhead của Mạch tổng quát:**** Nếu dùng zk-SNARKs (mạch tổng quát), ta phải chịu chi phí khởi tạo lớn (Trusted Setup, sinh Key), kích thước thư viện lớn, và thời gian tạo bằng chứng (Prover time) lâu hơn đáng kể do phải chuyển đổi đa thức đơn giản này thành hàng ngàn ràng buộc R1CS không cần thiết. Nó giống như việc dùng dao mổ trâu để giết gà.
- ****Tính đơn giản:**** Một giao thức Sigma chuyên biệt cho việc chứng minh kiến thức nghiệm của đa thức (dựa trên cam kết Pedersen chẳng hạn) sẽ rất nhanh, nhẹ, và dễ triển khai hơn nhiều cho trường hợp cụ thể này.

4 Ứng dụng thực tế (Real-World Applications)

4.1 (1) [10 điểm] Các hệ thống bảo vệ quyền riêng tư (Privacy-Preserving Systems)

Zcash và quyền riêng tư tài chính: Zcash sử dụng zk-SNARKs để cho phép giao dịch riêng tư trên chuỗi khối công khai.

4.1.1 (a) [5 điểm] Giải thích một giao dịch “shielded” trong Zcash chứng minh điều gì mà không tiết lộ điều gì

Trả lời:

Hãy nêu rõ những dữ liệu nào được giữ bí mật, và những dữ liệu nào được công khai.

Bài làm:

Trong Zcash, một giao dịch được bảo vệ (shielded transaction) sử dụng zk-SNARKs để chứng minh tính hợp lệ của việc chuyển tiền mà không để lộ thông tin nhạy cảm.

****Những điều được chứng minh (nhưng không tiết lộ chi tiết):****

- **Quyền sở hữu:**** Người gửi thực sự sở hữu các đồng coin đầu vào (tức là biết khóa bí mật chi tiêu tương ứng với cam kết trên blockchain).
- **Chống chi tiêu đôi (Double-spending):**** Các đồng coin này chưa từng được chi tiêu trước đó (thông qua cơ chế Nullifier duy nhất).
- **Bảo toàn giá trị:**** Tổng giá trị tiền đầu vào bằng tổng giá trị tiền đầu ra cộng với phí giao dịch (không có tiền nào được tạo ra từ hư không).

****Phân loại dữ liệu:****

• **Dữ liệu BÍ MẬT (được ẩn đi):**

- Địa chỉ người gửi (Sender address).
- Địa chỉ người nhận (Receiver address).
- Số tiền giao dịch (Amount).
- Khóa bí mật chi tiêu (Spending Key).

• **Dữ liệu CÔNG KHAI (trên Blockchain):**

- Bằng chứng zk-SNARK (Proof string).
- Các cam kết (Commitments) của các đồng coin mới được tạo ra (dưới dạng mã hóa).
- Nullifiers (các chuỗi số ngẫu nhiên đại diện cho việc hủy bỏ đồng coin cũ, để ngăn chặn việc dùng lại).
- Phí giao dịch (nếu là giao dịch minh bạch một phần).

4.1.2 (b) [5 điểm] Zcash đã tiến hóa từ Sprout → Sapling → Halo 2 (không cần thiết lập tin cậy)

Trả lời:

Giải thích vì sao việc loại bỏ “trusted setup” lại có ý nghĩa lớn đối với khả năng được chấp nhận của hệ thống.

Bài làm:

****Trusted Setup (Thiết lập tin cậy)**** là quy trình khởi tạo các tham số hệ thống cho zk-SNARKs đời đầu (như trong bản Sprout của Zcash). Nó yêu cầu một nhóm người tham gia tạo ra một “bí mật chung” (toxic waste) và sau đó phải tiêu hủy nó hoàn toàn.

****Rủi ro của Trusted Setup:**** Nếu nhóm người này thông đồng với nhau và giữ lại phần bí mật đó thay vì tiêu hủy, họ có thể tạo ra các bằng chứng giả mạo hợp lệ. Trong trường hợp của Zcash, điều này có nghĩa là họ có thể ****in tiền vô hạn**** mà không ai phát hiện ra, làm sụp đổ hoàn toàn nền kinh tế của đồng tiền này.

****Ý nghĩa của việc loại bỏ (trong Halo 2):**** Halo 2 sử dụng hệ thống chứng minh mới không cần Trusted Setup (gọi là Transparent Setup). Điều này có ý nghĩa cực lớn:

- ****Loại bỏ yếu tố con người:**** Không cần phải tin tưởng vào sự trung thực của bất kỳ cá nhân hay tổ chức nào trong lễ khởi tạo.
- ****An toàn tuyệt đối:**** Loại bỏ hoàn toàn nguy cơ tồn tại "backdoor" để in tiền giả.
- ****Tăng cường sự chấp nhận:**** Các tổ chức tài chính, chính phủ và người dùng phổ thông sẽ dễ dàng chấp nhận và tin tưởng sử dụng hệ thống hơn khi biết rằng sự an toàn của nó dựa hoàn toàn vào toán học công khai, chứ không phải dựa vào niềm tin vào một nhóm người lạ mặt.

4.2 Bonus Question (10 điểm cộng)

Bạn đang thiết kế một ví đa chữ ký (multi-signature wallet), trong đó bất kỳ 2 trong 3 bên đều có thể ủy quyền giao dịch.

Trả lời:

(a) Giải thích cách bạn có thể sử dụng chứng minh OR để tạo sơ đồ chữ ký ngưỡng (threshold signature)
Cụ thể, làm thế nào một tập con các bên ký có thể chứng minh họ cùng ký mà không tiết lộ bên nào thực sự đã ký?

Bài làm:

Để xây dựng chữ ký ngưỡng (Threshold Signature) bảo vệ quyền riêng tư cho cấu trúc truy cập "2 trong 3" (2-out-of-3), ta có thể sử dụng kỹ thuật ****OR Composition của các mệnh đề AND****.

Gọi K_1, K_2, K_3 lần lượt là các mệnh đề "Tôi biết khóa bí mật của người thứ 1, 2, 3". Cấu trúc "2 trong 3" có nghĩa là một trong các cặp sau đây đã ký: (1 và 2) HOẶC (1 và 3) HOẶC (2 và 3).

Ta xây dựng mệnh đề tổng hợp:

$$\text{Statement} = (K_1 \wedge K_2) \vee (K_1 \wedge K_3) \vee (K_2 \wedge K_3)$$

****Cách thực hiện:**** Người tạo chữ ký (ví dụ: nhóm gồm người 1 và người 3) sẽ:

- Thực hiện ****thật**** (Real Proof) cho phần $(K_1 \wedge K_3)$ vì họ có cả hai khóa a_1 và a_3 .
- Thực hiện ****mô phỏng**** (Simulated Proof) cho hai phần còn lại: $(K_1 \wedge K_2)$ và $(K_2 \wedge K_3)$ (vì họ thiếu a_2).
- Kết hợp lại bằng kỹ thuật OR (cộng các thách thức).

Kết quả là Verifier chỉ biết rằng "điều kiện 2-trong-3 đã được thỏa mãn", nhưng không thể biết chính xác cặp nào (1-2, 1-3 hay 2-3) đã thực hiện việc ký.

(b) Giả sử ba bên có khóa công khai

$$A_1 = g^{a_1}, \quad A_2 = g^{a_2}, \quad A_3 = g^{a_3}$$

Nếu bên 1 và 3 muốn cùng ký, hãy phác thảo giao thức Sigma mà họ sẽ sử dụng.

Bài làm:

Nếu bên 1 và 3 hợp tác để ký (họ biết a_1, a_3), giao thức sẽ diễn ra như sau:

1. ****Bước 1: Cam kết (Commitment)****

- Nhánh thật (1,3): Chọn y_1, y_3 ngẫu nhiên. Tính $Y_{1,3} = (g^{y_1}, g^{y_3})$.

- Nhánh giả (1,2) và (2,3): Chọn trước các thách thức giả $c_{1,2}, c_{2,3}$ và phản hồi giả. Tính ra các cam kết giả $Y_{1,2}$ và $Y_{2,3}$.
2. ****Bước 2: Thách thức (Challenge)**** Verifier gửi thách thức chủ C .
3. ****Bước 3: Phân phối thách thức & Phản hồi****
- Nhóm ký tính thách thức thật cho nhánh (1,3) là: $c_{1,3} = C - (c_{1,2} + c_{2,3}) \pmod{n}$.
 - Tính phản hồi thật cho nhánh (1,3):

$$r_1 = y_1 + c_{1,3} \cdot a_1, \quad r_3 = y_3 + c_{1,3} \cdot a_3$$

(Lưu ý: Trong nhánh AND, cả hai thành viên 1 và 3 phải dùng chung thách thức con $c_{1,3}$).

4. ****Bước 4: Gửi bằng chứng**** Gửi tất cả các cam kết, thách thức con và phản hồi cho Verifier.

(c) Thảo luận sự khác biệt giữa cách tiếp cận Zero-Knowledge này và các sơ đồ multi-signature truyền thống

Ưu điểm về quyền riêng tư mà bản ZK cung cấp là gì, và chi phí tính toán tăng thêm ra sao?

Bài làm:

- ****Multi-sig truyền thống (On-chain):**** Ví dụ như trong Bitcoin (P2SH hoặc 'OP_CHECKMULTISIG'), danh sách các khóa công khai tham gia ký thường được tiết lộ rõ ràng. Ai nhìn vào blockchain cũng biết chính xác "Ví A và Ví B đã ký giao dịch này". Không có sự riêng tư về danh tính người ký.
- ****ZK Multi-sig (Threshold ZK):****
 - ****Ưu điểm (Privacy):**** Cung cấp tính ẩn danh tuyệt đối cho tập hợp người ký (Signer Anonymity). Người ngoài chỉ biết "nhóm này đã đạt đồng thuận", không biết ai cụ thể trong nhóm đã bỏ phiếu thuận. Điều này quan trọng cho các tổ chức cần bảo mật quy trình ra quyết định nội bộ.
 - ****Chi phí (Cost):**** Đánh đổi lại, chi phí tính toán và kích thước dữ liệu lớn hơn nhiều.
 - * ****Tính toán:**** Phải thực hiện nhiều phép tính mũ (cho cả nhánh thật và nhánh giả), phức tạp hơn nhiều so với việc chỉ ký ECDSA đơn thuần.
 - * ****Kích thước:**** Bằng chứng ZK chứa nhiều giá trị (Y, c, r) cho mọi tổ hợp, tốn dung lượng lưu trữ hơn.